

JavaScript

Méthodes, fonctions fléchées et boucles

Des explications simples, un exemple pour chaque notion et les erreurs à éviter.

push()

ajouter une valeur

map()

transformer chaque valeur

filter()

garder certaines valeurs

sort()

trier le tableau

reduce()

obtenir un résultat final

=>

écrire une fonction courte

for...in

parcourir les clés

for...of

parcourir les valeurs

async

déclarer une fonction asynchrone

await

attendre le résultat d'une Promise

push(), map() et filter()

Trois méthodes très utilisées pour manipuler un tableau

push()

MODIFIE LE TABLEAU

À quoi ça sert ? Ajouter un ou plusieurs éléments à la fin d'un tableau.

CODE

```
const fruits = ["pomme", "banane"];
fruits.push("orange");
```

RÉSULTAT

```
["pomme", "banane", "orange"]
```

map()

NOUVEAU TABLEAU

Transforme chaque élément et retourne un nouveau tableau.

CODE

```
const nombres = [1, 2, 3];

const doubles = nombres.map(
  nombre => nombre * 2
);
```

```
console.log(doubles);
```

RÉSULTAT

```
[2, 4, 6]
```

Ici, map() passe sur chaque nombre et le multiplie par 2.

filter()

NOUVEAU TABLEAU

Garde seulement les éléments qui respectent une condition.

CODE

```
const ages = [12, 18, 25, 15];

const majeurs = ages.filter(
  age => age >= 18
);
```

```
console.log(majeurs);
```

RÉSULTAT

```
[18, 25]
```

La condition renvoie true pour 18 et 25 : ils sont conservés.

sort() et reduce()

Trier des valeurs et calculer un résultat final

sort()

[MODIFIE LE TABLEAU](#)

À quoi ça sert ? Trier les éléments. Pour des nombres, on donne une fonction de comparaison.

CODE

```
const nombres = [10, 3, 25, 1];  
  
nombres.sort((a, b) => a - b);
```

RÉSULTAT

[1, 3, 10, 25]

Comment comprendre a - b ?

Résultat négatif : a passe avant b. Résultat positif : b passe avant a.

reduce()

[UNE SEULE VALEUR](#)

À quoi ça sert ? Parcourir tout le tableau pour construire un seul résultat : une somme, un objet, un total, etc.

CODE

```
const prix = [10, 20, 5];  
  
const total = prix.reduce(  
  (accumulateur, valeur) => accumulateur + valeur,  
  0  
);
```

```
console.log(total);
```

RÉSULTAT

35

0 est la valeur de départ. À chaque tour, accumulateur garde le résultat précédent.

Arrow function =>

Le rôle de return dépend des accolades { }

La règle à retenir

Avec des accolades { } : il faut écrire return.

Sans accolades : le résultat est retourné automatiquement.

Avec accolades RETURN OBLIGATOIRE

On ouvre un bloc de code. JavaScript ne devine pas quelle valeur retourner.

CODE

```
const doubler = (nombre) => {  
  return nombre * 2;  
};
```

```
console.log(doubler(5));
```

RÉSULTAT

10

Sans accolades RETOUR AUTOMATIQUE

Quand il n'y a qu'une seule expression, son résultat est retourné automatiquement.

CODE

```
const doubler = nombre => nombre * 2;
```

```
console.log(doubler(5));
```

RÉSULTAT

10

Attention : parenthèses ≠ accolades

Les parenthèses () entourent souvent les paramètres. Ce sont les accolades { } qui changent la règle de return.

```
(a, b) => a + b           // retour automatique  
(a, b) => { return a + b; } // return obligatoire
```

for...in et for...of

L'une parcourt les clés, l'autre parcourt les valeurs

for...in

LES CLÉS

À utiliser surtout avec un objet. La variable reçoit le nom de chaque propriété.

CODE

```
const personne = {  
  nom: "Lina",  
  age: 25  
};  
  
for (const cle in personne) {  
  console.log(cle);  
  console.log(personne[cle]);  
}
```

RÉSULTAT

```
nom  
Lina  
age  
25
```

cle vaut d'abord "nom", puis "age".
personne[cle] donne la valeur.

for...of

LES VALEURS

À utiliser surtout avec un tableau. La variable reçoit directement chaque valeur.

CODE

```
const fruits = [  
  "pomme",  
  "banane",  
  "orange"  
];  
  
for (const fruit of fruits) {  
  console.log(fruit);  
}
```

RÉSULTAT

```
pomme  
banane  
orange
```

fruit contient directement la valeur du tableau à chaque tour.

Mémo très simple

for...in = je lis les étiquettes / les clés

for...of = je lis directement les valeurs

objet -> for...in tableau -> for...of

Pour parcourir un tableau avec son index, utilise plutôt une boucle for classique ou forEach().

async et await

Attendre une opération sans bloquer tout le programme

L'idée générale

Certaines opérations prennent du temps : récupérer des données, lire un fichier ou attendre une réponse. `async` et `await` permettent d'écrire ce code dans un ordre facile à lire.

async

PROMISE

`async` se place avant une fonction. La fonction retourne toujours une Promise, même si elle retourne une valeur simple.

CODE

```
async function obtenirNombre() {  
  return 42;  
}  
obtenirNombre().then(console.log);
```

RÉSULTAT

42

await

ATTENTE

`await` attend qu'une Promise soit terminée, puis récupère directement son résultat.

CODE

```
async function afficher() {  
  const resultat =  
    await Promise.resolve("OK");  
  console.log(resultat);  
}  
afficher();
```

RÉSULTAT

OK

Exemple complet avec `fetch()` et `try...catch`

TRY...CATCH

`fetch()` retourne une Promise. `await` attend la réponse. `try...catch` permet de gérer une erreur sans arrêter brutalement le programme.

CODE

```
async function chargerUtilisateur() {  
  try {  
    const reponse = await fetch("https://api.exemple.com/user");  
    const utilisateur = await reponse.json();  
  
    console.log(utilisateur.nom);  
  } catch (erreur) {  
    console.log("Erreur de chargement");  
  }  
}  
chargerUtilisateur();
```

Ordre de lecture :

1. lancer la demande 2. attendre la réponse 3. convertir les données 4. utiliser le résultat

À retenir : `await` s'utilise généralement à l'intérieur d'une fonction `async`.

Les Promises

Une valeur qui sera disponible plus tard

L'idée générale

Une Promise représente un résultat que JavaScript n'a pas encore. Elle peut réussir ou échouer plus tard.

pending

EN ATTENTE

L'opération est encore en cours. Le résultat n'est pas disponible.

fulfilled

RÉUSSIE

L'opération a réussi. La Promise contient maintenant une valeur.

rejected

ÉCHOUÉE

L'opération a échoué. La Promise contient une erreur.

Le cycle d'une Promise

pending

->

fulfilled

ou

rejected

Une fois terminée, elle ne change plus.

Exemple simple

Au départ, la Promise est pending. Après 100 ms, `resolve()` la fait passer à fulfilled.

CODE

```
const promesse = new Promise((resolve, reject) => {  
  setTimeout(() => {  
    resolve("Trajet trouve");  
  }, 100);  
});  
  
promesse.then((résultat) => console.log(résultat));
```

then() utilise la valeur si la Promise est fulfilled. catch() gère une Promise rejected.

Promise.all()

Lancer plusieurs Promises en même temps

À quoi ça sert ?

Promise.all() lance plusieurs opérations en parallèle et attend qu'elles soient toutes terminées.

Exemple avec le trajet

CODE

```
const [trajet, listePassagers] = await Promise.all([
  getTrajet(id),
  getPassagers(id),
]);
```

Les résultats gardent le même ordre que les Promises dans le tableau.

L'un après l'autre

ENVIRON 200 MS

CODE

```
const trajet = await getTrajet(id);
const passagers = await getPassagers(id)
```

Le deuxième appel démarre après le premier.

En parallèle

ENVIRON 100 MS

CODE

```
const resultats = await Promise.all([
  getTrajet(id), getPassagers(id)
]);
```

Les deux appels démarrent tout de suite.

À retenir

1. On utilise Promise.all() quand les opérations ne dépendent pas les unes des autres.
2. await attend que toutes les Promises soient terminées.
3. Si une Promise est rejected, Promise.all() est aussi rejected.